

Federated Meta-Learning Based Computation Offloading Approach With Energy-Delay Tradeoffs in UAV-Assisted VEC

Chunlin Li , Chaoyue Deng, Yong Zhang, and Shaohua Wan , *Senior Member, IEEE*

Abstract—Federated learning (FL) provides an applicable solution for computation offloading in Unmanned Aerial Vehicle(UAV)-assisted Vehicular Edge Computing (VEC) by preserving privacy. However, the heterogeneity of clients brings challenges to the generalization of models. Therefore, we propose a federated meta-learning (FML) framework to solve computation offloading for UAV-assisted VEC. In this paper, we are concerned with computation offloading of temporary hotspot regions due to traffic congestion. First, we construct a computation offloading problem with energy-delay tradeoffs and convert the problem to a Markov Decision Process (MDP). Then, we use FML to train personalized models for different vehicles while enhancing the generalization, we propose a Graph neural network-based FL Probabilistic Embedding for Actor-critic RL (GFL-PEARL) algorithm. We model the context as a Directed Acyclic Graph (DAG) and use GNN to reconstruct the inference network of the PEARL algorithm to extract the correlation between contexts fully. We dynamically adjust the task priority during the FML training process to improve the sampling efficiency. Finally, we verify the performance of the algorithm through simulation and physical experiments. Experimental results show that our algorithm can reduce average cost and task overtime rate by 31 % and 56 % respectively compared with the benchmarks.

Index Terms—Computation offloading, UAV-assisted VEC, energy-latency tradeoffs, GFL-PEARL, FML.

I. INTRODUCTION

A. Background and Motivations

THE rapid development of intelligent connected vehicles has brought about intelligent vehicle applications such as

autonomous driving, path planning, and road condition prediction. These applications rely on technologies such as high-precision positioning and radar sensing and have extremely strict requirements on computing resources and latency. In addition, real-time analysis of driver behavior can provide early warning services for drivers, which also has extremely stringent requirements on low latency. However, the computing power of terminal vehicles is limited, and it is difficult to meet the demands of these computationally intensive and latency-sensitive tasks. For this reason, vehicle edge computing (VEC) has become an ideal solution for computation offloading in the Internet of Vehicles (IoV). VEC shortens response time and improves quality of service by deploying edge servers on roadside units (RSUs) close to the road.

When RSU and nearby vehicles are overloaded or inaccessible, such as temporary congestion and RSU failure, the offloading task cannot be received and completed within the delay requirement, which reduces the quality of experience for drivers [1]. Compared with fixed RSUs, unmanned aerial vehicle (UAV), with the high mobility, flexible deployment and strong adaptability, has been employed as an optional candidate to receive offloading tasks from vehicles and provide flexible offloading services in VEC. It provided an economical and efficient computation paradigm known as UAV-assisted VEC. Recent studies [2], [3] in the academic community have explored the potential of UAVs in VEC, even considering these limitations. With the developments of UAV battery and computing power, UAVs have substantial potential as a offloading platform and should be further explored.

Computation offloading has become a key means to solve the performance bottlenecks of resource-constrained devices. Unlike traditional distributed training methods, federated learning (FL)-based offloading approaches reduce communication overhead and achieve better data privacy protection [27]. However, in UAV-assisted VEC, the global model is difficult to consider the personalized needs of different vehicles, and exhibits poor generalization ability in the scenarios with large differences in data distribution and diverse tasks [28]. Different from FL, federated meta-learning (FML) improves the efficiency of computation offloading by rapidly adapting to new tasks, implementing personalized optimization strategies, and facilitating cross-device collaborative computing [30]. However, there are still some challenges to be addressed.

Received 7 April 2024; revised 15 May 2025; accepted 21 May 2025. Date of publication 23 May 2025; date of current version 3 September 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62372344, Grant 62171330, and Grant 62172438, in part by the National Key R&D Program of China under Grant 2023YFB3308701, in part by the Key Research and Development Plan of Hubei Province under Grant 2023BAB075, in part by the International Science and Technology Cooperation Project of Hubei Province under Grant 2024EHA042, in part by Wuhan Key RD Program Projects under Grant 2024050702030091, in part by Shenzhen Science and Technology Program under Grant JCYJ20220818103200002, and in part by the Natural Science Foundation of Guangdong Province of China under Grant 2024A1515011155. Recommended for acceptance by M. Segata. (Corresponding author: Chunlin Li.)

Chunlin Li, Chaoyue Deng, and Yong Zhang are with the School of Computer and Artificial Intelligence, Wuhan University of Technology, Wuhan 430063, China (e-mail: chunlinli2020@163.com).

Shaohua Wan is with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518110, China (e-mail: shaohua.wan@uestc.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3573278

- How to perform efficient computation offloading under limited communication bandwidth and computing resources in UAV-assisted VEC?
- How to improve the generalization and robustness of the model while addressing potential security and privacy concerns of the training phase in complex and dynamic UAV-assisted VEC?

With the challenges mentioned above, this work focuses on solving the computation offloading problem in UAV-assisted VEC using an FML framework. In this framework, although some studies [16], [18] have considered the trade-off between energy consumption and latency in offloading, to the best of our knowledge, no study has exhaustively considered the total energy consumption of the system. Although location and trajectory optimizations of UAV have been discussed in previous studies [16], [17], [18], [19], [20], there is no work to consider the RSSI optimization in UAV deployment. In current works on computation offloading optimization, most existing studies rely on simple linear or independent assumptions when dealing with constraints between tasks, and do not fully consider complex dependencies between tasks, resulting in poor adaptability of the model in dynamic environments. In addition, task sampling strategy of traditional methods is usually statically allocated, and the sampling priority cannot be flexibly adjusted, which limits the speed of model convergence.

B. Contributions

The main contributions and innovations are as follows:

- 1) *FML-based Computation Offloading Framework for UAV-assisted VEC*: In this framework, local model on vehicle-side only needs to upload the intermediate parameters or gradients without sharing the vehicle data, which protects user privacy during the training phase. We consider offloading tasks to nearby vehicles, UAVs and edge nodes to fully utilize computing resources and improve task execution efficiency. Then, we built a UAV deployment problem with goal of maximizing Received Signal Strength Indication (RSSI), which effectively improve the communication quality between UAVs and vehicles. We consider the transmission and execution energy consumption, the hovering and flight energy consumption of UAVs, and establish an optimization problem with energy consumption-latency tradeoffs. Experimental results show that when number of vehicles is 50, GFL-PEARL reduces task timeout rate by up to 31.72% than baselines.
- 2) *GFL-PEARL Algorithm for Computation Offloading Optimization*: In the GFL-PEARL algorithm, the context that refers to task constraints is modeled as Directed Acyclic Graph (DAG) and we reconstruct the reasoning network using Graph Neural Network (GNN) to fully extract the correlation between contexts, so the model can quickly adapt to different tasks, thereby improving robustness. Meanwhile, through information sharing and variational inference between tasks, it can better capture the uncertainty in task distribution, reduce overfitting, and thus improve the generalization of the model. In addition, we

TABLE I
COMPARISON OF OUR APPROACH WITH RELATED WORKS

Reference	[4], [5], [6], [7]	[8], [9], [10], [11]	[12], [13], [14], [15]	[16], [17], [18], [19], [20]	Our work
Edge-only offloading	✓				
Nearby vehicles-only offloading		✓			
Collaborative offloading			✓	✓	✓
UAV-assisted offloading				✓	✓
RSSI optimization					✓
UAV energy consumption				✓	✓
System energy consumption					✓
Energy-latency tradeoffs				✓	✓
Offloading mode	full offloading	full offloading	partial offloading	partial full offloading	full offloading

dynamically adjust the task sampling priority to accelerate the convergence speed. Experimental results show that compared with baselines, GFL-PEARL algorithm only needs 300 iterations before the reward tends to a stable value, which significantly improves computation offloading efficiency in dynamic traffic scenario.

The rest of paper are organized as follows. Section II introduces related works. Section III presents the system model and framework. Section IV presents the algorithms. Section V provides experimental results. Section VI concludes the paper.

II. RELATED WORK

In VEC, when RSU computing resources are scarce, optimizing server deployment and utilizing idle devices are efficient solutions to alleviate server computing overload [19]. However, above two solutions cannot avoid RSU overload or unaccessed. When the vehicle needs to handle complex and computationally intensive tasks such as driver behavior detection, due to the limited computing resources of the vehicle, it cannot perform all tasks independently, so some tasks need to be offloaded to RSU or nearby vehicles for execution. In this section, we review the related works including computation offloading in VEC, and solutions for computation offloading.

A. Computation Offloading Strategy

Computation offloading addresses computation overload by transferring vehicle-side tasks from resource-constrained devices to edge nodes or cloud, reducing local burden and improving efficiency. The comparison of proposed computation offloading scheme with related works is shown in Table I.

1) *Edge-Only Offloading*: To improve the efficiency of computation offloading, Abadi et al. [4] proposed a Deep Reinforcement Learning (DRL) algorithm to optimize computing and network resources in edge layer. Liang et al. [5] proposed a double deep Q-network for single-task source nodes and multi-task destinations, which. Yan et al. [6] utilized Q-Learning and DRL for multi-task source nodes and single-task destinations, optimizing offloading and resource allocation to improve task execution efficiency. Shi et al. [7] presented deep deterministic policy gradient algorithms for edge offloading in multi-user scenarios.

2) *Nearby Vehicles-Only Offloading*: Chen et al. [8] introduced a computation offloading scheme in vehicle-mounted networks that enhances efficiency by offloading tasks to nearby vehicles, thereby reducing both load and latency. Chen et al. [9]

proposed a decentralized computation offloading mechanism that facilitates efficient coordination of offloading among mobile users. Huang et al. [10] developed a deep reinforcement learning framework aimed at optimizing offloading and resource allocation, demonstrating adaptability in dynamic environments. Zhang et al. [11] presented a dynamic offloading scheme that integrates vehicle-to-vehicle collaboration with energy collection to minimize energy consumption and improve computational efficiency.

3) *Collaborative Offloading*: Peng et al. [12] reformulated computation offloading and resource allocation as reinforcement learning problems, solved using distributed deep learning algorithms. Bi et al. [13] utilized RSUs and vehicle resources for task offloading in the Internet of Vehicles, enabling vehicles to also participate in data relay and computation. Zhang et al. [14] introduced a DRL algorithm that combines mobile edge computing nodes, base stations, vehicles, and RSUs to minimize energy consumption and transmission delays. Lastly, Wang et al. [15] proposed a vehicle-coordinated offloading strategy, allowing vehicles to migrate unfinished tasks to others before leaving traffic infrastructure coverage.

4) *UAV-Assisted Offloading*: In temporary hotspot scenarios, UAV-assisted vehicle networks have been explored for computation offloading. Zhao et al. [16] examined a UAV-assisted vehicle edge computing network to minimize mobile device energy consumption while meeting latency requirements. Feng et al. [17] combined UAV-assisted vehicle networks with machine learning to enhance communication efficiency and adaptability. Hu et al. [18] proposed a strategy where UAVs facilitate data transmission and provide wireless charging, improving offloading capability and energy efficiency. Ning et al. [19] optimized UAV flight trajectories to enhance energy collection and reduce latency, stabilizing computation offloading. Finally, Zhang et al. [20] aimed to maximize system throughput by dynamically adjusting UAV flight paths and resource allocation, thereby improving overall performance.

In summary, above offloading schemes provide an effective solution for improving the execution efficiency of vehicle tasks. However, [4], [5], [6], [7], [8], [9], [10], [11] are more suitable for relatively static scenes, and do not work well in dynamic traffic environments. [12], [13], [14], [15] ignore the special cases of RSU unaccessed or vehicle overload, which are not suitable for the proposed scenario. [16], [17], [18], [19], [20] introduce UAVs into special scenarios for computation offloading, but they rarely consider RSSI optimization in UAV deployment, leading to cannot guarantee the link quality between UAV and vehicles. Finally, [16], [17], [18], [19], [20] only consider the UAV energy consumption instead of whole energy consumption, resulting in a decrease in overall system energy efficiency. Different from the above mentioned studies, we built a UAV deployment problem with goal of maximizing RSSI, aiming to improve the communication quality between UAVs and vehicles. We formulate the computation offloading problem as total energy consumption and latency trade-off problem, with the goal of minimizing the service delay and maximizing the system energy efficiency.

TABLE II
COMPARISON OF OUR APPROACH WITH RELATED WORKS

Reference	[21], [22], [23]	[24], [25], [26]	[27], [28], [29]	[30], [31]
Mathematical Programming	✓			
Deep Learning		✓		
Reinforcement Learning		✓		
Meta-Reinforcement Learning			✓	✓
Federated Learning				
Method	IPSGA Estimation, Ant colony	DRL, DDPG, Q-learning, DCOM	FL	Pearl

B. Solution for Computation Offloading

Previous studies have investigated different solutions for computation offloading in MEC environments, There are basically included mathematical optimization, deep learning, federated learning, and meta-reinforcement learning. The comparison of the proposed computation offloading algorithm with related works is shown in Table II.

1) *Mathematical Programming*: Li et al. [21] proposed an improved particle swarm genetic algorithm (IPSGA) to solve the offloading decision. Shan et al. [22] proposed a load balance-based computing offloading framework to obtain offloading strategies through covariance-based hierarchical analysis method and potential game. Chuang et al. [23] proposed a real-time, two-stage ant colony algorithm.

2) *Deep Learning and Reinforcement Learning*: To improve generalization, deep learning-based offloading algorithms have emerged. Geng et al. [24] proposed a multi-intelligent DRL computing offloading framework with improved actor and criterion networks for feature extraction. Li et al. [25] proposed a distributed-based DRL method and solved the offloading decision by an improved D3QN. Zhao et al. [26] proposed an optimization framework based on multi-agent DRL to minimize latency and energy consumption in UAV-assisted mobile edge computing.

3) *Federated Learning*: In the dynamic Internet of Vehicles, extensive state representation can hinder training speeds. Federated learning approaches have been explored. Hou et al. [27] introduced a dual-time scale DRL model for varied delay tasks, optimizing offloading and resource usage. Tang et al. [28] proposed a model-free distributed DRL algorithm for independent offloading strategies, reducing delays and abandonment rates. Li et al. [29] proposed an improved federated deep deterministic policy gradient (DDPG) algorithm to solve the computing offloading problem by improving the exploration noise and the experience pool.

4) *Meta-Reinforcement Learning*: The PEARL algorithm [30], a context-based meta-reinforcement learning method, has gained traction. The FedMetaR2Ag algorithm [31] is a meta-reinforcement learning algorithm that obtains satisfactory results with a small number of samples and iterations.

In summary, above works efficiently solves the optimization problem of offloading with the mathematical optimization, deep learning, and reinforcement learning schemes. However, [20], [21], [22] are usually based on static parameters, which is not flexible in dynamic UAV assisted scenarios. [23], [24], [25], [26], [27], [28] take a long time to converge to the optimal

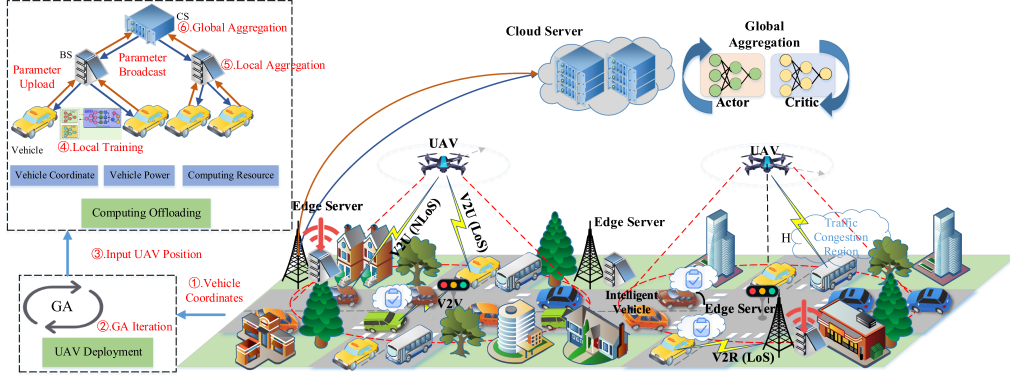


Fig. 1. UAV-assisted VEC FML framework.

strategy, which affects the real-time requirements. [29], [30] rarely consider their intrinsic connections when processing context, thus limiting the generalization ability of the model across different tasks. In view of the above limitations, we propose an GFL-PEARL algorithm, which improves the generalization and robustness of the model, reduce average cost and task timeout rate.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Overview

Fig. 1 shows the proposed FML framework in UAV-assisted VEC, which includes four roles: vehicles, RSU with edge server (ES), UAVs, and cloud server (CS). Each RSU is equipped with an ES, therefore, in the subsequent content, RSU refers to the RSU and the equipped ES. We describe the workflow of the framework: Step 1 inputs the coordinates of all the vehicles in a certain time slot. Step 2 obtains the optimal position of the UAV through the GA algorithm by iterating continuously. Step 3 inputs the optimal position into the computation offloading problem. The step 4-step 6 indicates the flow of computation offloading. Table III shows the definition of key symbols.

B. Federated Meta-Learning Framework

In the proposed framework, the clients of FML consist of RSUs and vehicles. Before each training round, each vehicle downloads the parameters from the server. After a training period, each RSU performs local aggregation, and then upload the parameters and losses to the cloud server for global aggregation. The training goal for FML are as follows:

$$\min L(\zeta) = \frac{1}{|I|} \sum_{i=1}^{|I|} l_i(\zeta - \alpha \nabla l_i(\zeta)). \quad (1)$$

where $|I|$ is the number of RSUs, l_i is the loss function for the parameter ζ , α is the connection ratio.

Each RSU uploads its gradient and meta loss to the cloud server at the end of the training phase. The update is done as follows:

$$\zeta^t = \zeta^{t-1} + \sum_{i=1}^{|I|} \omega_i \Delta \zeta_i^t, \quad (2)$$

TABLE III
SYMBOL DEFINITION

Symbol	Definition
k, m	Request vehicle k , Service vehicle m
u, i	UAV u , RSU i
Co_k, Co_i, Co_u	Device's coordination
$d_{k,i}^{v2r}, d_{k,m}^{v2u}, d_{k,u}^{v2u}$	Distance between devices
$r_{k,m}^{v2v}, r_{k,i}^{v2r}, r_{k,u}^{v2u}$	Transmission rate between devices
K, U, I	The set of vehicle, UAV, RSU
$R_i^v = \{1, 2, \dots, R_i^v \}$	The set of vehicles within the coverage of RSU i
$f_{k,\max}, f_{u,\max}, f_{i,\max}$	Max Computation resource
$f_{k,u}, f_{k,i}, f_{k,m}$	Computation resource allocated to vehicle k
$p_{k,u}^{trans}, p_{k,i}^{exec}$	Transmission and executing power of vehicle k
Q_k, S_k	Computation tasks and task size of vehicle k
u_k	Resource required for vehicle k unit sized tasks
v_k	The velocity of vehicle k
$t_{k,\max}$	Maximum tolerance time of vehicle k
t_{con}	Communication time
τ	Path loss exponent
$T_{k,u}^{local}, E_{k,u}^{local}$	Local delay, energy consumption
$T_{k,u}^{trans}, E_{k,u}^{trans}$	Transmission delay, energy consumption
$E_{k,u}^{hover}, E_{k,u}^{fly}$	Hover and fly energy consumption
d_{max}^u	Maximum distance of a single movement
R	Radius of coverage of the UAV
W	Channel bandwidth
$\kappa_{LoS}, \kappa_{NLoS}$	Excessive path loss

$$\omega_i = \frac{c_i + \lambda \cdot d_i}{\sum_{j=1}^{|I|} c_j + \lambda \cdot d_j}, c_i = \frac{|D_i|}{\sum_{j=1}^{|I|} |D_j|}, d_i = \frac{\left(\sum_{k=1}^{|K|} f_k^i \right)^\beta}{\sum_{j=1}^{|I|} \left(\sum_{k=1}^{|K|} f_k^j \right)^\beta}, \quad (3)$$

where ζ^t is the parameters at the t -th training round, ω_i is the aggregation weights, D_i is the amount of data and f_k^i is the meta loss, λ and β are the aggregation parameters.

C. UAV Deployment Model

We assume that the UAV is flying at a fixed altitude and that its state can be viewed as stationary at time slot t . Fixing the flight altitude of UAVs can simplify the deployment problem while avoiding unnecessary energy costs associated with the frequent ascent and descent of UAVs [2]. The coordinate of the UAV u is $Co_u(t) = (x_u(t), y_u(t), H)$, H is the fixed altitude at which UAVs can fly. At the next time slot, the coordinate of the UAV u is $Co_u(t+1) = (x_u(t+1), y_u(t+1), H)$. Therefore, we can

get the following reasonableness constraints:

$$0 \leq \|Co_u(t+1) - Co_u(t)\| \leq d_{\max}^u, t \in T, \quad (4)$$

where $d_{\max}^u$ is the maximum distance traveled by the UAV in a single movement.

Because the UAV can fly within a specific rectangular area with x-axis range $[0, x_{\max}]$ and y-axis range $[0, y_{\max}]$ so that the coordinates of the UAVs satisfy the following constraints:

$$0 \leq x_u \leq x_{\max}, 0 \leq y_u \leq y_{\max}. \quad (5)$$

In order to ensure that the UAV can finally return for a successful periodic flight, we give the following constraints:

$$Co(T+1) = Co(1). \quad (6)$$

Similar to [32], we ensure that the coverage of UAVs does not overlap in order to avoid UAV collisions and reduce interference between UAVs. Therefore the distance between UAVs needs to satisfy the following constraint:

$$\|Co_{u'}(t) - Co_u(t)\| \geq R, u' \neq u, \quad (7)$$

where R is radius of coverage of UAV.

We adjust the position of UAVs based on the Received Signal Strength Indicator (RSSI) [33]:

$$RSSI_{u,k} = p_k^{trans} - 20 \log(d_{k,u}^{v2u} f_c 4\pi/c), \quad (8)$$

where p_k^{trans} is the transmission power of the vehicle k , f_c is the carrier frequency, $d_{k,u}$ is the distance between vehicle k and UAV u , and c is the velocity of light.

Within the communication coverage area of the UAV u , each vehicle may establish a connection with UAV u . Therefore, the optimization objective is as follows:

$$\mathbf{P1} : \text{Max} \quad \frac{1}{N_u^v} \sum_{k=1}^{N_u^v} RSSI_{u,k}, \quad (9)$$

$$\text{s.t.} \quad (4)-(7), \quad (9a)$$

where N_u^v is the number of vehicles in the Communication coverage area of the UAV u .

D. Computing Offloading Model in UAV-Assisted VEC

In time slot t , vehicle k generate a driving behavior recognition task $Q_k \triangleq \{v_k, t_{k,\max}, S_k, u_k\}$. Vehicle k will only select a device for offloading that has the best quality of communication with it. We set the variable x_k^{v2x} to indicate whether vehicle chooses to offload the task to device x . Therefore, the variables must satisfy the following:

$$x_k^{local} + x_k^{v2r} + x_k^{v2u} + x_k^{v2v} = 1. \quad (10)$$

It is assume that the coordinate of the vehicle k is $Co_k = (x_k, y_k, 0)$, and it travels at v_k speed along the θ_k direction. The coordinate of the other device e is $Co_e = (x_e, y_e, h_e)$, the speed is v_e along the θ_e direction. Therefore, the distance in the y-axis direction is $d_y = y_k - y_e$, and the velocity difference is $v_{k,e,y} = v_k \cos \theta_k - v_e \cos \theta_e$. The x-axis distance is $d_x = x_k - x_e$, and the velocity difference is $v_{k,e,x} = v_k \sin \theta_k - v_e \sin \theta_e$. Therefore, the distance between vehicle k and device e must satisfy

the following constraint:

$$(d_y + v_{k,e,y} t_{con})^2 + (d_x + v_{k,e,x} t_{con})^2 + h_e^2 \leq d_{\max}^2. \quad (11)$$

Since the result after the execution of the task is smaller than the task itself, we ignore the return delay [14]. Our system employs a binary offloading model [34], [35], meaning tasks are either fully processed locally or offloaded to external entities, such as UAVs, RSUs, or other nearby vehicles. The system needs to select the optimal offloading decision, and we propose a federated meta-learning algorithm to optimize the offloading decisions. The detailed operations for different computation modes, based on the decisions derived from the federated meta-learning algorithm, are as follows.

1) *Locally Executing*: When a task is executed locally, the execution delay and execution energy consumption of vehicle k are calculated as follows:

$$T_k^{exec} = \frac{N_k}{f_k^{exec}}, \quad (12)$$

$$E_k^{exec} = p_k^{exec} T_k^{exec}, \quad (13)$$

where $N_k = u_k \times S_k$, f_k^{exec} is the allocated computation resource for executing task of vehicle k . Since vehicles need to participate in the training process of FML, the FML training delay and training energy consumption of vehicle k are as follows:

$$T_{k,train}^{FML} = \frac{D_k \times u_k}{f_k^{train}}, \quad (14)$$

$$E_{k,train}^{FML} = p_k^{exec} T_{k,train}^{FML}, \quad (15)$$

where D_k is the local dataset size of vehicle k and f_k^{train} is the allocated computation resources for training of vehicle k .

Therefore, the total delay and total energy consumption of vehicle k are calculated as follows:

$$T_k^{local} = T_k^{exec} + T_{k,train}^{FML}, \quad (16)$$

$$E_k^{local} = E_k^{exec} + E_{k,train}^{FML}. \quad (17)$$

2) *Offloading to UAVs*: When offloading to UAV, the communication mode is divided into LoS and NLoS. The LoS link and NLoS link probability are calculated as follows:

$$Pr_{k,u}^{LoS} = \left(1 + \mu_a \exp \left(-\mu_b \left(\arcsin \left(\frac{H}{d_{k,u}^{v2u}} \right) - \mu_a \right) \right) \right)^{-1}, \quad (18)$$

$$Pr_{k,u}^{NLoS} = 1 - Pr_{k,u}^{LoS}, \quad (19)$$

where μ_a and μ_b are environmental constants.

The path loss consists of free space path loss and excessive path loss, calculated as follows:

$$L_{k,u}^{LoS} = 20 \log(d_{k,u}^{v2u} f_c 4\pi/c) + \kappa_{LoS}, \quad (20)$$

$$L_{k,u}^{NLoS} = 20 \log(d_{k,u}^{v2u} f_c 4\pi/c) + \kappa_{NLoS}, \quad (21)$$

where κ_{LoS} and κ_{NLoS} correspond to the excessive path loss of the different communication modes, respectively.

Therefore, the average path loss [18] and signal-to-noise ratio (SNR) ratio are calculated as follows:

$$\overline{L_{k,u}} = Pr_{k,u}^{LoS} L_{k,u}^{LoS} + Pr_{k,u}^{NLoS} L_{k,u}^{NLoS}, \quad (22)$$

$$\omega_{k,u} = \frac{p_k^{trans} 10^{-\overline{L_{k,u}}/10}}{\sigma^2 + \sum_{j=1, j \neq k}^V p_j^{trans} 10^{-\overline{L_{j,u}}/10}}, \quad (23)$$

where σ^2 is the Gaussian noise power. The transmission rate between vehicle k and UAV u can be calculated as follows:

$$r_{k,u}^{v2u} = W_{k,u} \log_2(1 + \omega_{k,u}), \quad (24)$$

where $W_{k,u}$ is channel bandwidth between vehicle k and UAV u . During the vehicle transmission task to UAV, the transmission delay and energy consumption are as follows:

$$T_{k,u}^{trans} = \frac{S_k}{r_{k,u}^{v2u}}, \quad (25)$$

$$E_{k,u}^{trans} = p_k^{trans} T_{k,u}^{trans}. \quad (26)$$

The execution delay and execution energy consumption of the UAV are calculated as follows:

$$T_{k,u}^{exec} = \frac{N_k}{f_{k,u}^{v2u}}, \quad (27)$$

$$E_{k,u}^{exec} = \omega^{uav} (f_{k,u}^{v2u})^3 T_{k,u}^{exec}, \quad (28)$$

where $f_{k,u}^{v2u}$ is the allocated computation resource by UAV u to vehicle k . ω^{uav} is the energy coefficient of the UAV [2], which is related to the chip structure.

The UAV is assumed to hover for a finite amount of time during transmission and execution. Therefore, the hovering energy consumption is calculated as follows:

$$E_u^{hover} = \psi_u (T_{k,u}^{trans} + T_{k,u}^{exec}), \quad (29)$$

where ψ_u [17] is a constant energy value.

The flying energy consumption is calculated as follows:

$$E_u^{fly} = \varpi (||Co_u(t+1) - Co_u(t)||/\Delta t)^2, \quad (30)$$

where ϖ [18] is related to the weight of the UAV.

Therefore, the total delay and total energy consumption of UAV are calculated as follows:

$$T_{k,u}^{v2u} = T_{k,u}^{trans} + T_{k,u}^{exec}, \quad (31)$$

$$E_{k,u}^{v2u} = E_{k,u}^{trans} + E_{k,u}^{exec} + E_u^{hover} + E_u^{fly}. \quad (32)$$

3) *Offloading to RSUs*: When offloading to RSU, the transmission rate between vehicle k and RSU i is as follows:

$$r_{k,i}^{v2r} = W_{k,i} \log_2 \left[1 + \frac{(d_{k,i}^{v2r})^{-\tau} p_k^{trans}}{\sigma^2 + \sum_{j=1, j \neq k}^V (d_{j,i}^{v2r})^{-\tau} p_j^{trans}} \right], \quad (33)$$

where τ is the path loss exponent.

In the process of FML, the vehicle needs to upload the locally trained model to the RSUs. The FML parameter upload delay of vehicle k is calculated as follows:

$$T_{k,up}^{FML} = \frac{Q}{r_{k,i}^{v2r}}, \quad (34)$$

where Q is the size of the upload model parameter.

The FML parameter upload delay depends on the maximum value of the upload delay, so the upload delay and upload energy consumption are calculated as follows:

$$T_{up}^{FML} = \max\{T_{k,up}^{FML}\}, k \in K, \quad (35)$$

$$E_{up}^{FML} = p_k^{trans} T_{up}^{FML}. \quad (36)$$

Therefore, the total delay and energy consumption of RSU are calculated as follows:

$$T_{k,i}^{v2r} = T_{k,i}^{trans} + T_{k,i}^{exec} + T_{up}^{FML}, \quad (37)$$

$$E_{k,i}^{v2r} = E_{k,i}^{trans} + E_{k,i}^{exec} + E_{up}^{FML}, \quad (38)$$

the rest of the equation is calculated similarly to (12), (13), (25), and (26).

4) *Offloading to Other Vehicles*: When offloading to the other vehicle, the transmission rate is calculated as follows:

$$r_{k,m}^{v2v} = W_{k,m} \log_2 \left[1 + \frac{(d_{k,m}^{v2v})^{-\tau} p_k^{trans}}{\sigma^2} \right]. \quad (39)$$

Therefore, the total delay and total energy consumption of task processing vehicle m are calculated as follows:

$$T_{k,m}^{v2v} = T_{k,m}^{trans} + T_{k,m}^{exec}, \quad (40)$$

$$E_{k,m}^{v2v} = E_{k,m}^{trans} + E_{k,m}^{exec}, \quad (41)$$

the rest of the equation is calculated similarly to (12), (13), (25), and (26).

Therefore, the total delay and total energy consumption of the vehicle k executing task Q_k are calculated as follows:

$$T_k = x_k^{local} T_k^{local} + x_k^{v2r} T_{k,i}^{v2r} + x_k^{v2u} T_{k,u}^{v2u} + x_k^{v2v} T_{k,m}^{v2v}, \quad (42)$$

$$E_k = x_k^{local} E_k^{local} + x_k^{v2r} E_{k,i}^{v2r} + x_k^{v2u} E_{k,u}^{v2u} + x_k^{v2v} E_{k,m}^{v2v}. \quad (43)$$

The optimization goal and constraints of the computation offloading problem of UAV-assisted VEC are as follows:

$$\mathbf{P2} : \min_{X,F} \left(\frac{1}{\sum_{i=1}^I R_i^v} \sum_{i=1}^I \sum_{k=1}^{R_i^v} (\kappa_t T_k + \kappa_e E_k) \right), \quad (44)$$

$$\text{s.t.} \quad \kappa_t + \kappa_e = 1, \quad (44a)$$

$$x_k^{local}, x_k^{v2r}, x_k^{v2u}, x_k^{v2v} \in \{0, 1\}, \quad (44b)$$

$$x_k^{local} + x_k^{v2r} + x_k^{v2u} + x_k^{v2v} = 1, \forall k \in K, \quad (44c)$$

$$\sum_{k=1}^K f_{k,u} \leq f_{u,\max}, \sum_{k=1}^K f_{k,i} \leq f_{i,\max}, \sum_{k=1}^K f_{k,m} \leq f_{m,\max}, \quad (44de)$$

$$T_k \leq t_{k,\max}, T_k \leq t_{\text{con}}, \quad (44e)$$

where (44a) indicates the sum of the weights is 1, (44b) and (44c) indicate that a task can choose only one offloading mode. (44d) indicate the sum of the computing resources allocated cannot exceed the maximum amount. (44e) indicates that the

total delay does not exceed the maximum tolerable time, and the total delay does not exceed the connection delay.

IV. PROBLEM SOLUTION AND ALGORITHM DESCRIPTION

A. Outline of Proposed Solution

The goal of **P1** is to optimize the deployment position of the UAV by maximizing RSSI. By optimizing the deployment position of the UAV, the data transmission rate can be greatly improved. The purpose of **P2** is to reduce the system energy consumption and latency of the vehicle to process all computation tasks under acceptable delay constraints. Reasonable deployment of the UAV position during computation offloading can ensure the communication quality of vehicle and reduce the latency and energy consumption of completing the task by reducing the communication cost between the UAV and the vehicle. For **P1**, we use the GA to solve for the optimal deployment coordinates of the UAV in different time slots (Algorithm 1). The **P2** has constraints on continuous variables, such as the coordinates Co_u of the UAV, and constraints on discrete variables, such as x_k^{local} . Also, it has nonlinear constraints such as constraint (44e). Therefore, the **P2** is a mixed-integer nonlinear programming (MINLP) problem, and it is difficult to find the optimal solution in polynomial time [36]. However, the **P2** can be modeled as an MDP $M = (S, A, P, R)$. In our GFL-PEARL algorithm, the positions and velocities of UAVs and vehicles, as well as the positions of RSUs, are used as state inputs for the neural network. By learning from these input features, the neural network can capture various factors affecting communication quality, including the impact of the distance between devices on RSSI. This approach enables the system to address signal fluctuations in complex urban environments, ensuring successful task completion and connectivity for full task offloading.

1) *State Space*: At the time slot t , the state space can be expressed as $S_t = \{H_t, CO_t, F_t\}$, where $H_t = \{h_1, h_2, \dots, h_K\}$, $h_k = \{Q_k, p_k, v_k, \theta_k\}$. $CO_t = \{Co_k, Co_u, Co_i\}$. $F_t = \{F_k, F_u, F_i\}$ is the computing resource set.

2) *Action Space*: In time slot t , the action space a_t is defined as $a_t = \{X_k, F_k\}, \forall k \in K$, where $X_k = \{x_k^{local}, x_k^{v2r}, x_k^{v2u}, x_k^{v2v}\}$ and $F_k = \{f_k^{local}, f_{k,i}^{v2r}, f_{k,u}^{v2u}, f_{k,m}^{v2v}\}$ denote the offloading and resource allocation decision, respectively.

3) *Reward Function*: The optimization objective is to minimize the weighted delay and energy consumption. Therefore, the reward function is as follows:

$$R(s_t, a_t) = - \sum_{i=1}^I \sum_{k=1}^{R_i^v} (\kappa_t t_k + \kappa_e E_k). \quad (45)$$

The state space and optimization objective are coupled with the coordinates of the UAV. Therefore, after obtaining the UAV coordinates, we input them into **P2** and use the GFL-PEARL algorithm to solve offloading strategy (Algorithm 2).

B. UAV Deployment Optimization Algorithm

In the GA, the cost function is same as (9). In initialization phase, the population is generated randomly, and the fitness

Algorithm 1: UAV Deployment Optimization Based on GA.

Input: Number of iterations $Iter$, population size

$PopSize$, Co_k , lower

bound Pos_Min , upper bound Pos_Max .

Output: UAV's optimal positions.

1: **for** $i=1:PopSize$ **do**

2: $pos_i = rand(Pos_Min, Pos_Max)$

3: Calculating $RSSI_i$

4: **end for**

5: **for** $k=1:Iter$ **do**

6: **for** $i=1:PopSize$ **do**

7: Chromosome crossover, mutation

8: Get next generation $pos_i = pos_{i+1}$

9: Calculating $RSSI_i$

10: $CurrBestRSSI_i = Max(RSSI_i)$

11: $CurrBestpos_i = pos_i$

12: **end for**

13: $GlobalBestRSSI_i = Max(CurrBestRSSI_i)$

14: $GlobalBestBestpos_i = Max(CurrBestpos_i)$

15: **end for**

16: **return** UAV's optimal positions

function is same as (8). The algorithm's pseudo-code is shown in Algorithm 1. The main time expense of the Algorithm 1 is incurred by the two-level loop structure, so the time complexity of UAV trajectory optimization algorithm based on genetic algorithm is $O(Iter \times Popsiz)$, where $Iter$ is the number of iterations and $Popsiz$ is the population size.

C. GFL-PEARL-Based Computation Offloading

The PEARL algorithm introduces a task belief Z to help agents make reasonable decisions, so its policy function is $\pi_\theta(a_t|s_t, z)$. PEARL learns and inference Z by variational inference methods [37], and its core process is defining and training an inference network $q_\eta(z|c)$. The variational lower bound on the objective function is:

$$E_T[E_{z \sim q_\eta(z|c^T)}[R(T, z) + \beta D_{KL}(q_\eta(z|c^T) || p(z))]], \quad (46)$$

where $R(T, z)$ is the objective function, and $p(z)$ is the prior Gaussian distribution of Z . According to the replacement invariance, $q_\eta(z|c)$ is denoted as:

$$q_\eta(z | c_{1:N}) \propto \prod_{n=1}^N \mathcal{N}(f_\eta^u(c_n), f_\eta^\sigma(c_n)), \quad (47)$$

where f_η is a neural network with parameter η . Network $f_\eta^u(c_n)$ estimates the mean. Network $f_\eta^\sigma(c_n)$ estimates the variance. The PEARL loss function is as follows:

$$L_{actor} = E_{\substack{(s_t, a_t) \sim D \\ a \sim \pi_\theta \\ z \sim q_\eta(z|c)}} \left[D_{KL} \left(\pi(a_t | s_t, \bar{z}) \left\| \frac{\exp(Q^\pi(s_t, a_t, \bar{z}))}{Z^\pi(s_t)} \right\| \right) \right], \quad (48)$$

$$L_{critic} = E_{\substack{(s_t, a_t) \sim D \\ z \sim q_\eta(z|c)}} \left[\frac{1}{2} (Q_\theta(s_t, a_t, z)) \right]$$

$$- (R(s_t, a_t) + \gamma E_{s'_t \sim p}[V_{\bar{\theta}}(s'_t, \bar{z})])^2 \Big], \quad (49)$$

$$L_{KL} = \beta D_{KL}(q(z|c)||p(z)). \quad (50)$$

For meta-learning, multiple tasks are faced; hence, meta-loss is used to update the parameters. The meta-loss is as follows:

$$f^\theta = \sum_{n=1}^N L_{critic}^n, f^\phi = \sum_{n=1}^N L_{actor}^n, f^\eta = \sum_{n=1}^N (L_{critic}^n + L_{KL}^n), \quad (51)$$

where M is the size of a batch of tasks.

We model context c as a DAG and reconstruct the inference network using the graph network. For a sampling process c^i during training, we model it as $C^i = (c^i, g^i)$, c^i containing several transitions. g^i corresponds to the adjacency matrix of these transitions. If there is s_{t_1+1} equal to s_{t_2} for the transition $(s_{t_1}, a_{t_1}, r_{t_1}, s_{t_1+1})$ and $(s_{t_2}, a_{t_2}, r_{t_2}, s_{t_2+1})$ in c^i , then the value of the corresponding position (t_1, t_2) in g^i will be 1.

$$g^i(t_1, t_2) = \begin{cases} 1 & \text{if } s_{t_1+1} = s_{t_2} \\ 0 & \text{otherwise.} \end{cases} \quad (52)$$

The node features in the DAG, i.e., the features F_i of the state transfer process (s_i, a_i, r_i, s_{i+1}) and the adjacency matrix of the directed graph, are inputted into the GNN to obtain the high-dimensional features of the node F'_i :

$$atten_{ij} = \frac{\exp(\text{LeakyReLU}(a^T[W F_i || W F_j]))}{\sum_{k \in Node_i} \exp(\text{LeakyReLU}(a^T[W F_i || W F_k]))}, \quad (53)$$

$$F'_i = \prod_{k=1}^K \text{Sigmoid} \left(\sum_{j \in Node_i} atten_{ij}^k W^k F_j \right), \quad (54)$$

where $atten_{ij}$ is the attention coefficient, representing the weight of the state transfer process t_j in the set of neighboring nodes of t_i , $Node_i$ denotes the set of neighboring points of t_i . After obtaining F'_i , it is inputted into the encoder network constructed through LSTM according to the processing order of DAG nodes. The encoding result depends on its characteristics and the previous state, and the encoding result is denoted as $ec = [ec_1, ec_2, \dots, ec_n]$.

$$ec_i = \text{encoder}(F'_i, ec_{i-1}). \quad (55)$$

After obtaining the encoding of each transfer process, we define the input of the decoder as follows:

$$dc_i = \text{decoder}(ct_i, dc_{i-1}, a_{i-1}), \quad (56)$$

$$ct_i = \sum_{j=0}^n \alpha_{ij} ec_j, \quad (57)$$

$$\alpha_{ij} = \frac{\exp(\text{eval}(dc_{i-1}, ec_j))}{\sum_{k=1}^n \exp(\text{eval}(dc_{i-1}, ec_k))}, \quad (58)$$

where ct_i is the feature vector of the i -th step, α_{ij} is the attention coefficient, $eval$ is a network for evaluating the degree of match

between the output at the i -th position and the input at the j -th position. The final output is into a three-layer MLP using the Softmax function.

$$A_{DAG} = \text{softmax}(\text{input}). \quad (59)$$

We improve the sampler of the PEARL algorithm by proposing a reward-based prioritization sampler RS_c , making the algorithm more likely to reach those highly rewarded states [38]. Therefore, we use a simple way to prioritize sampling:

$$\bar{r}_t = e^{\mu r(s_t, a_t)} + \varepsilon, \quad (60)$$

where $\bar{r}_t$ is the sampling priority of transition $(s_t, a_t, r(s_t, a_t), s'_t)$, μ is a constant greater than 0, and ε constant greater than 0 to prevent a situation where the priority tends to zero. After the introduction of priority sampling, due to greedy strategy of intelligent body, it may lead to some transitions with low priority not being sampled all the time, thus falling into local optimum or even divergence. It is necessary to introduce sampling weights, before which sampling probability of a sample is

$$PR_t = \frac{\bar{r}_t}{\sum_s \bar{r}_s}. \quad (61)$$

According to the sampling probability, the sampling weight of the sample is defined as:

$$W_t = \frac{[N_{\text{sample}} \times PR_t]^{-\beta}}{\max_s \{W_s\}}, \quad (62)$$

where $\beta \in (0, 1]$ is used to control the number, N_{sample} is the number of samples, and when calculating W_t , it is necessary to replace W_s with $\bar{r}_s$. Algorithm 2 show the pseudo-code, and the computation offloading framework based on FML is shown in Fig. 2.

Complexity analysis: The GFL-PEARL based computational offloading algorithm requires a total of $O(It)$ iterations, during each iteration, the RSU obtains the global parameters and sends them down to the vehicles in the coverage area, hence the time complexity is $O(It * (I + K))$. Then, each vehicle needs to collect data for different meta-tasks to populate the experience playback pool, hence the time complexity is $O(N * T * P)$, where P is a model parameter. Finally, each vehicle needs to select data from the playback pool to update the model parameters in reverse, with a time complexity of $O(S * N * P)$. Without considering aggregation and uploading, the total time complexity is about $O(It * (I + K) * (T + S) * N * P)$.

D. Offloading Decision Algorithm

As shown in Algorithm 3, after the training stage, an offloading decision-making algorithm will be executed. Let n represent the number of iterations and m represent the number of steps in each iteration. We assume that the complexity of the sampling operation is $O(p)$, where p is the number of sampled data points. The complexity of the action selection operation is $O(a)$, where p is the size of the action space. The complexity of

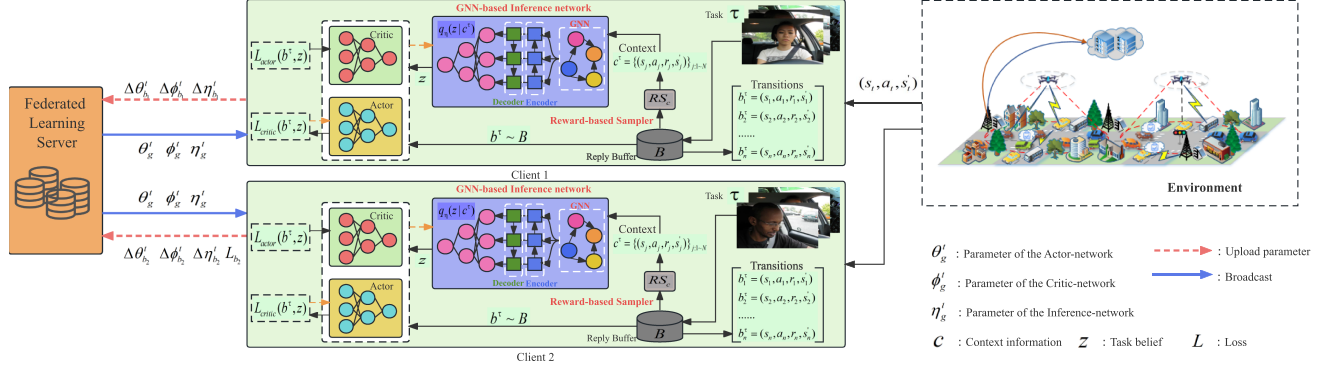


Fig. 2. The computation offloading framework based on FML.

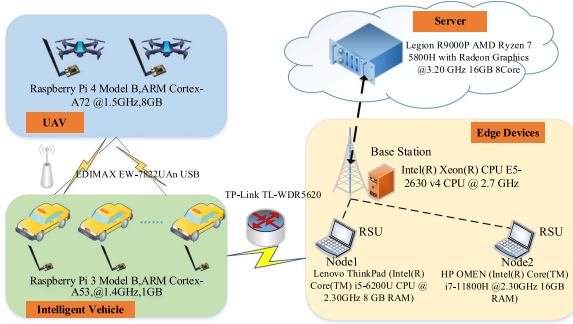


Fig. 3. UAV-assisted VEC experimental framework.

executing the action and obtaining the reward is $O(e)$, and the complexity of the state update operation is $O(s)$. Therefore, the overall complexity of Algorithm 3 can be expressed as $O(n \times m \times (p + a + e + s))$.

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Setup

To verify the UAV deployment algorithm proposed in this paper, we used OMNet+SUMO to simulate real traffic scenarios. In details, the OMNet is a network simulator that simulates wireless network communication, and SUMO is a traffic simulator that generates real network scenarios and is available to other programs. The main parameters of the experiment are shown in Table IV.

To verify the computation offloading algorithm proposed in this paper, we built a laboratory-level drone-assisted vehicle-mounted edge computing platform, and the experiment framework is shown in Fig. 3, where the UAV carries a Raspberry Pi to provide computing resources for the requesting vehicle. We use the classical DNN model YOLOv2 to recognize drivers' driving behaviors and perform computation offloading experiments using the trained model. The experimental datasets are the YawDD [40] and Kaggle State Farm [41] datasets. These datasets contain images and videos of abnormal driving behaviors. To protect the user privacy, these images are preprocessed locally on the device, such as through feature extraction [42], before being used for further analysis. The main parameters of

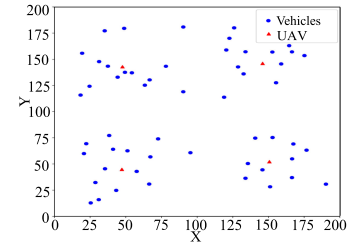


Fig. 4. UAV deployment.

the simulation experiment are shown in Table V. The experiment results shown are the averages of 30 simulations, together with 95 percent confidence intervals.

To measure the performance of the algorithms, we choose average delay, average energy consumption, average cost, and task timeout rate as evaluation metrics. We choose IGPSO [13], DCOM [24], FedMetaR2Ag [31] and FedAvg [46] as comparison algorithms. IGPSO algorithm combines the diversity of GA algorithms with the fast convergence of PSO algorithms, thus has a better performance compared to traditional heuristic algorithms. DCOM is a multi-agent reinforcement learning algorithm, which improves the actor-critic network structure by considering the correlation between experiences. FedMetaR2Ag combines FL with the MAML algorithm and aggregates gradients by averaging losses. Both of them are similar to the proposed algorithm in terms of optimization objective, algorithmic type, or application scenarios, but are essentially different. Hence, we choose them as benchmarks.

B. Results and Analysis

1) *The Simulation Results of UAV Deployment*: As shown in Fig. 4, we set up four UAVs and fifty vehicles and iteratively solve optimal deployment location of UAVs by GA algorithm. It can be seen that the four UAVs all provide good coverage for ground vehicles in the area, providing communication and computing services for the vehicles.

2) *The Experimental Results of Computation Offloading*: This study is conducted under following scenario: when RSU is overloaded or unavailable, the frequency of offloading to UAVs increases; conversely, the usage frequency of UAVs is lower.

Algorithm 2: GFL-PEARL-Based Computation Offloading.

Input: Training rounds It ; Co_u ; Batch of training task $\{\tau_i\}_{i=1\dots N}$; Learning Rate $\lambda_1, \lambda_2, \lambda_3$; Time Step T ; Training Step S .

Output: Global parameters for pre-training θ^g, ϕ^g , and η^g .

```

1: for  $s = 1, 2, \dots, It$  do
2:   for Each RSU  $i, i \in I$  do
3:      $i$  gets parameters  $\theta^i \leftarrow \theta^g, \phi^i \leftarrow \phi^g, \eta^i \leftarrow \eta^g$ 
4:     for Each vehicle  $k, k \in K$  do
5:        $\theta^k \leftarrow \theta^i, \phi^k \leftarrow \phi^i, \eta^k \leftarrow \eta^i$ 
6:       for each  $\tau_i$  do
7:         Initialize replay buffer  $\mathcal{D}_i, c^i = \{\}$ .
8:         for  $t = 1, \dots, T$  do
9:           Sampling  $z \sim q_\eta(z|c^i)$ .
10:           $a_t \sim \pi_\theta(a|s, z), r_t \sim R(s, a)$ .
11:           $s'_t \sim p(s'_t|s_t, a_t), (s_t, a_t, r_t, s'_t) \rightarrow \mathcal{D}_i$ .
12:           $c^i = \{(s_t, a_t, s'_t, r_t)\}_{t:1\dots T} \sim \mathcal{D}_i$ .
13:        end for
14:      end for
15:    for step in training steps do
16:      for each  $\tau_i$  do
17:         $c^i \sim RS_c(\mathcal{D}_i), (s, a, r, s')^i \sim \mathcal{D}_i$ .
18:        Sample  $z \sim q_\eta(z|c^i)$ .
19:        Calculate  $L_{actor}^i, L_{critic}^i, L_{KL}^i$ 
20:      end for
21:      Calculate  $f^\theta, f^\phi, f^\eta, \theta \leftarrow \theta - \lambda_1 \nabla_\theta f^\theta$ .
22:       $\phi \leftarrow \phi - \lambda_2 \nabla_\phi f^\phi, \eta \leftarrow \eta - \lambda_3 \nabla_\eta f^\eta$ .
23:    end for
24:    The  $k$  uploads meta-losses to  $i$ .
25:  end for
26:   $i$  calculates loss and uploads parameter and loss.
27: end for
28: The cloud server aggregate globally (2).
29: end for
30: return global network parameters  $\theta^g, \phi^g$  and  $\eta^g$ .

```

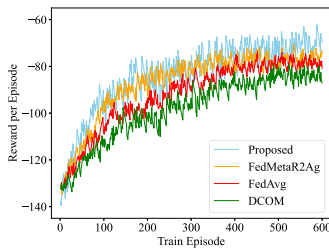


Fig. 5. Convergence performance.

Fig. 5 shows the convergence performance of all algorithms. From Fig. 5, we can see that GFL-PEARL is better than the benchmark algorithm and FedAvg. GFL-PEARL converges after about 300 iterations and is faster compared to DCOM and FedAvg, because meta-learning is able to adapt to dynamic and complex VEC environments and train personalized models for different users in the face of different data distributions. Moreover, it performs better than FedMetaR2Ag, because we

Algorithm 3: Offloading Decision.

Input: Global parameters θ^g, ϕ^g and η^g , the number of iterations K , the number of steps per iteration T .

Output: Offloading and Allocation Strategies X^*, F^* .

```

1: for each episode  $k = 1, 2, \dots, K$  do
2:   Initialize networks and get the state  $s_t$ .
3:   for each slot  $t = 1, 2, \dots, T$  do
4:     Sample  $z \sim q_\eta(z|c^k)$  by inference network.
5:     Get the action  $a_t$  from  $\pi_\theta(a|s, z)$ .
6:     Execute the action  $a_t$  and get the reward  $r_t$ .
7:     Update status  $s'_t$ .
8:   end for
9: end for
10: return  $X^*, F^*$ .

```

TABLE IV
UAV DEPLOYMENT PARAMETERS

Parameter	Value
The intelligent vehicle's amount (K)	50
The UAV's amount (U)	4
The vehicle's transmission power (p_k^{trans})	30dBm
Carrier frequency f_c	2GHz
The Maximum UAV's speed [39]	15m/s
The maximum distance d_{max}^u [39]	30m
boundary x_{max}, y_{max}	200m, 200m
Coverage radius R	45m
The probability of mating,mutating	0.5, 0.2
The number of generations	100

TABLE V
COMPUTATION OFFLOADING PARAMETERS

Parameter	Value
The amount of vehicle(K)	[10, 50]
The amount of UAV(U) and RSU(I)	4, 2
Total Channel bandwidth W	10MHz
The vehicle's CPU frequency $f_{k,max}$	2GHz
The CPU frequency of UAV $f_{u,max}$ and RSU $f_{i,max}$	1.5GHz, 4GHz
The task's Size (S_k)	[4.4, 6.8] Mbit
The task's volume ration (u_k)	50cycles/bit
FML aggregation parameter [43] λ, β	10, 1.5
FML connection ratio α	0.8
The learning rate [31] $\lambda_1, \lambda_2, \lambda_3$	0.003
The SAC target smoothing coefficient [44]	0.005
The UAV's capacitance coefficient [5] (ω^{uav})	10^{-28}
The task's maximum tolerable time	[1.5, 5.0]s
The task batch size M	128
The UAV's hovering power ψ_u [2]	80W
Training epoch	600
Encoder time-steps [44]	64
attenuation factor $\kappa_{LoS}, \kappa_{NLoS}$ [45]	1.7, 24
Gaussian Noise power σ^2	-110dBm
Environmental constants μ_a, μ_b	11.61, 0.12
The UAV's flying height H	60m
The time slot t	0.5s

use a graph neural network to better capture the correlation between experiences. In conclusion, our proposed GFL-PEARL algorithm is more suitable for dynamic and complex in-vehicle edge computing scenarios.

As shown in Fig. 6, with the number of iterations increases, the performance of the four algorithms improves. When the number of iterations reaches 600, in terms of task timeout rate, the GFL-PEARL algorithm is 87.7% lower than the IGPSO

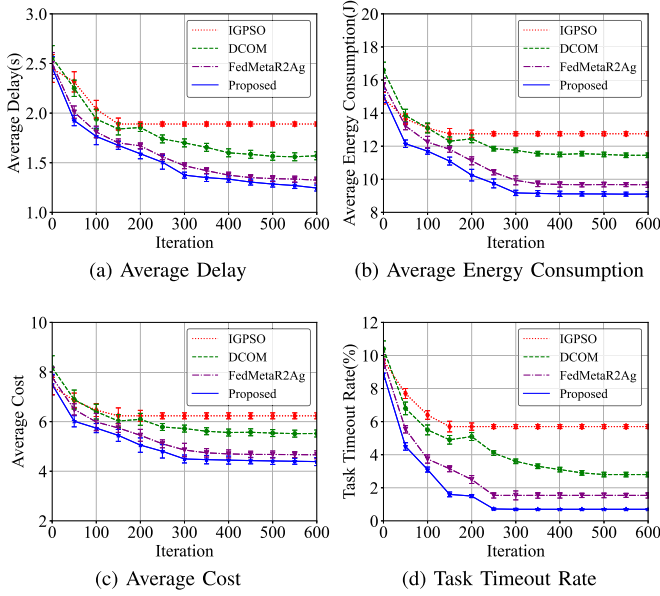


Fig. 6. The impact of the number of iterations on performance.

algorithm, 75% lower than the DCOM algorithm, and 54.5% lower than the FedMetaR2Ag algorithm. When the number of iterations reaches about 150, the IGPSO algorithm falls into a local optimum. The proposed algorithm is more effective than the DCOM algorithm because the deterministic policy in the DCOM algorithm is not conducive to action exploration and may also fall into local optimality. The proposed algorithm has a better performance compared to the FedMetaR2Ag algorithm, which is based on the SAC algorithm, which uses stochasticity policy, which outputs the policy in the algorithm in a way that not only ensures that the rewards are maximized but also ensures that the entropy of the information is maximized, and thus is able to explore more space than the FedMetaR2Ag algorithm.

In Fig. 7, with the increasing number of vehicles, all metrics of different algorithms are increasing. When number of intelligent vehicles reaches 50, in terms of task timeout rate, the GFL-PEARL algorithm is 31.72% lower than the IGPSO algorithm, 22.03% lower than the DCOM algorithm, and 12.39% lower than the FedMetaR2Ag algorithm. As the number of intelligent vehicles increases, the IGPSO algorithm cannot reasonably unload tasks that cannot be handled locally by the vehicles, resulting in these tasks being unable to be completed within the tolerable time. The deterministic gradient of the DCOM algorithm limits its exploration ability. The FedMetaR2Ag algorithm combines the characteristics of meta-reinforcement learning, and its performance is similar to that of the algorithm in this paper. The proposed algorithm is based on the SAC algorithm, which takes into account the information entropy of the strategy and has a strong ability to explore strategies. It is not easy to fall into the local optimum when the number of intelligent vehicles increases and the Markov space becomes complex. The metrics of the proposed algorithm increase smoothly with minimal fluctuations, indicating that the system exhibits good stability. Moreover, as the number of vehicles increases from 10 to 50, the task timeout rate remains below 18%.

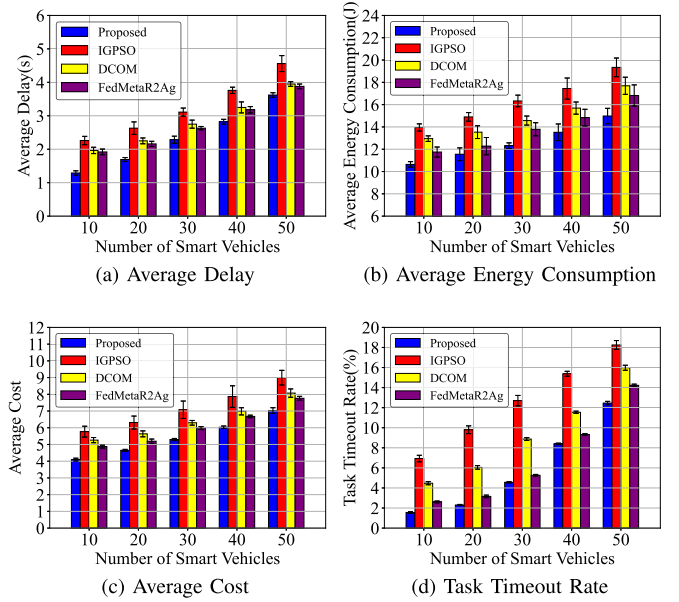


Fig. 7. The impact of the number of intelligent vehicles on performance.

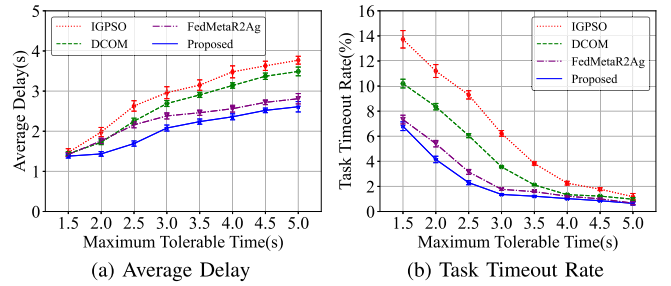


Fig. 8. The impact of the maximum tolerable time on performance.

Fig. 8 shows that as the maximum tolerable time increases, the average delay of tasks completed within the tolerable time also gradually increases. When the tolerable time is 1.5 s, the average delay of tasks completed within the tolerable time of the four algorithms is similar. Because tolerable time is short, to complete the task more quickly, more computing resources need to be allocated, and computing resources are short. Hence, tasks completed on time are close to the completion time limit. As the tolerable time increases, the computing resources that need to be allocated to process each task decrease, and the performance gap of the algorithm begins to appear. When the maximum tolerable time reaches 5 s, the GFL-PEARL algorithm is 30.76% lower than the IGPSO algorithm, 25.21% lower than the DCOM algorithm, and 9.34% lower than the FedMetaR2Ag algorithm in terms of average time delay. In addition, our algorithm takes approximately 0.57 milliseconds to make a decision on the RSU server, which is far below the maximum tolerance time.

As shown in Fig. 9, the average cost of different algorithms and the task timeout rate continue to increase as the task size increases. This is because the increase in the task size leads to an increase in the computing resources required to process the task, and hence, the cost and task timeout rate required to process the task increases continuously. As the task size increases, the

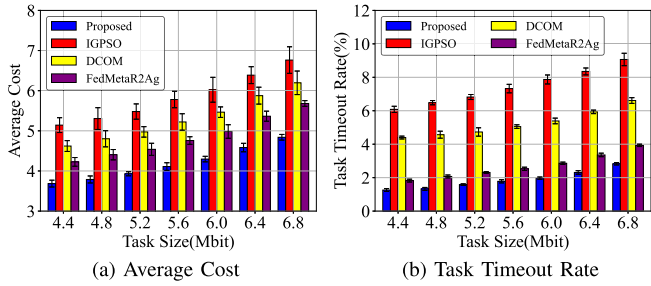


Fig. 9. The impact of the task size on performance.

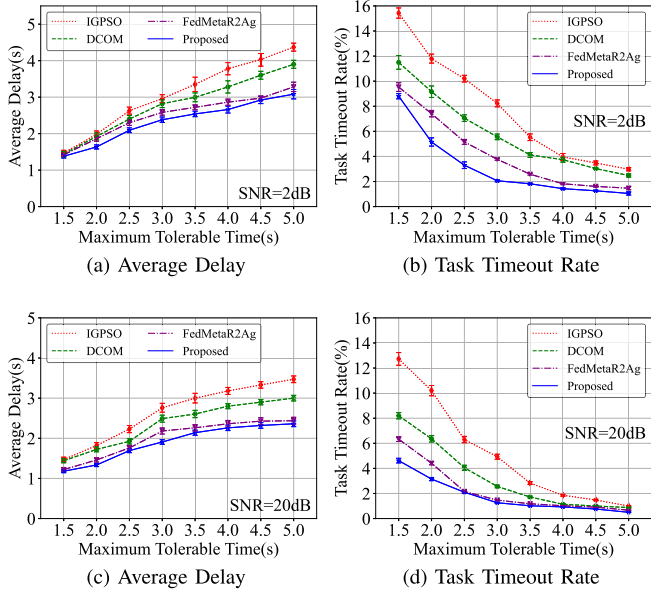


Fig. 10. The impact of the maximum tolerable time at different SNR.

IGPSO algorithm adapts poorly to changes in the environment and has the largest increase in average cost and task timeout. When the task size reaches 6.8 Mbit, the GFL-PEARL algorithm is 28.45% lower than the IGPSO algorithm, 21.9% lower than the DCOM algorithm, and 14.82% lower than the FedMetaR2Ag algorithm in terms of average cost.

We added Gaussian noise to the wireless channel with different SNR values to study the performance under different channel conditions. Fig. 10 shows the impact of the maximum tolerance time on the release delay and the task completion rate under two-channel conditions (SNR = 2 dB or 20 dB). When the channel condition is poor, the average delay and task timeout rate of the four algorithms increase. This is because when the power of the vehicle transmitting data remains unchanged, the poor channel condition will lead to a decrease in the transmission rate, thus increasing the offload delay. Under a certain maximum tolerance time, the increase in the offloading delay will inevitably lead to an increase in the task timeout rate. When channel conditions are poor, the task timeout rate of the algorithm proposed in this paper is reduced by 56.3%, 49.3% and 16.6% compared to the IGPSO, DCOM and FedMetaR2Ag algorithms. The algorithm proposed in this paper is relatively less affected by changes in channel conditions. This is because the algorithm is based on the SAC algorithm, which ensures the maximization of information

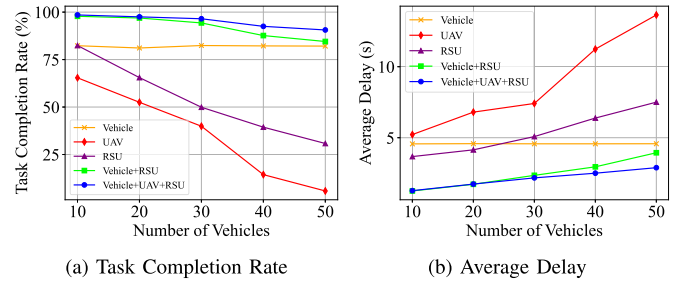


Fig. 11. UAV ablation study: Task completion rate and average delay.

entropy. Therefore, it has better adaptability to environmental changes than the other three algorithms.

To verify the behavior and usage frequency of UAVs in the system, we conducted an ablation study, as shown in Fig. 11. In both the full UAV and full RSU offloading scenarios, task completion rates dropped sharply. In the scenarios without UAVs and in the scenario with UAVs, the task completion rate gradually decreased with increase in vehicle numbers. When number of vehicles is 50, the task completion rate with UAVs was 90.6%, compared to only 84.52% without UAVs. This trend indicates that as vehicle numbers grow, UAV usage frequency increases, helping alleviate high RSU loads. Regarding task latency, the latency for full local vehicle computing remained relatively stable, around 4.57 seconds. The full UAV offloading scenario saw a latency increase of nearly 198% with 50 vehicles, while the full RSU offloading scenario experienced a 64% latency increase. In vehicle-RSU collaboration scenario, latency showed a 72% reduction compared to all-local vehicle computing. Although the computing power of UAVs is inferior compared to vehicles, when the RSU load is high, they are as candidate options, considerably improving task completion rate and reducing delay.

VI. CONCLUSION AND FUTURE WORKS

In this paper, we analyze the computation offloading problem in UAV-assisted VEC. We built a computation offloading model with goal of minimizing the system delay and energy consumption. Besides, we also constructed a UAV movement model to dynamically adjust the UAV's deployment position to be more realistic. Finally, we convert the offloading decision problem into an MDP problem and solve it using GFL-PEARL algorithm. The experimental results verify the superiority of GFL-PEARL algorithm compared with the benchmark algorithm. In the future, we will focus on extending the framework to multi-UAV cooperative scenarios and incorporating more sophisticated energy harvesting and security mechanisms to further enhance system performance and resilience. In the future, we will focus on extending the framework to multi-UAV cooperative scenarios and incorporating more sophisticated energy harvesting and security mechanisms to further enhance system performance and resilience.

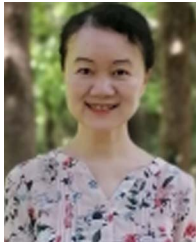
REFERENCES

- [1] Z. Han, Y. Yang, W. Wang, L. Zhou, T. N. Nguyen, and C. Su, "Age efficient optimization in UAV-aided VEC network: A game theory viewpoint," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 25287–25296, Dec. 2022.
- [2] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.
- [3] C. Yang, B. Liu, H. Li, B. Li, K. Xie, and S. Xie, "Learning based channel allocation and task offloading in temporary UAV-assisted vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 9965–9977, Sep. 2022.
- [4] J. K. Abadi, Z. Zahra, N. Mansouri, and M. M. Javidi, "Deep reinforcement learning-based scheduling in distributed systems: A critical review," *Knowl. Inf. Syst.*, vol. 66, pp. 5709–5782, 2024.
- [5] S. Liang, H. Wan, T. Qin, J. Li, and W. Chen, "Multi-user computation offloading for mobile edge computing: A deep reinforcement learning and game theory approach," in *Proc. IEEE Int. Conf. Commun. Technol.*, 2020, pp. 1534–1539.
- [6] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.
- [7] B. Shi, Y. Pan, and L. Huang, "Deep reinforcement learning based task offloading and resource allocation strategy across multiple edge servers," *Service Oriented Comput. Appl.*, vol. 14, no. 1, pp. 1–14, 2024.
- [8] R. Chen, Y. Fan, S. Yuan, and Y. Hao, "Vehicle collaborative partial offloading strategy in vehicular edge computing," *Mathematics*, vol. 12, no. 10, May 2024, Art. no. 1466.
- [9] X. Chen, "Decentralized computation offloading game for mobile cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 4, pp. 974–983, Apr. 2015.
- [10] Y. Huang et al., "Multi-agent-deep-reinforcement-learning-enabled offloading scheme for energy minimization in vehicle-to-everything communication systems," *Electronics*, vol. 13, no. 3, Feb. 2024, Art. no. 663.
- [11] J. Zhang, H. Guo, and J. Liu, "Adaptive task offloading in vehicular edge computing networks: A reinforcement learning based scheme," *Mobile Netw. Appl.*, vol. 25, no. 5, pp. 1736–1745, 2020.
- [12] H. Peng and X. Shen, "Deep reinforcement learning-based resource management for multi-access edge computing in vehicular networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2416–2428, Fourth Quarter 2020.
- [13] J. Bi, H. Yuan, S. Duanmu, M. Zhou, and A. Abusorrah, "Energy optimized partial computation offloading in mobile-edge computing with genetic simulated-annealing-based particle swarm optimization," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3774–3785, Mar. 2021.
- [14] H. Zhang, X. Liu, Y. Xu, D. Li, C. Yuen, and Q. Xue, "Partial offloading and resource allocation for MEC-Assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1276–1288, Jan. 2024.
- [15] J. Wang, J. Hu, G. Min, W. Zhan, Q. Ni, and N. Georgalas, "Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 64–69, May 2019.
- [16] P. Zhao, Z. Kuang, Y. Guo, and F. Hou, "Task offloading and resource allocation in UAV-assisted vehicle platoon system," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 1584–1596, Jan. 2025.
- [17] G. Feng, C. Wang, and B. Li, "UAV-assisted wireless relay networks for mobile offloading and trajectory optimization," *Peer-to-Peer Netw. Appl.*, vol. 12, no. 6, pp. 1820–1834, Nov. 2019.
- [18] Q. Hu, Y. Cai, G. Yu, Z. Qin, M. Zhao, and G. Y. Li, "Joint offloading and trajectory design for UAV-enabled mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1879–1892, Apr. 2019.
- [19] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.
- [20] H. Zhang, W. Wu, C. Wang, M. Li, and R. Yang, "Deep reinforcement learning-based offloading decision optimization in mobile edge computing," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2019, pp. 1–7.
- [21] C. Li, L. Chai, K. Jiang, Y. Zhang, J. Liu, and S. Wan, "DNN partition and offloading strategy with improved particle swarm genetic algorithm in VEC," *IEEE Trans. Intell. Veh.*, early access, Dec. 25, 2023, doi: 10.1109/ITV.2023.3346506.
- [22] N. Shan, Y. Li, and X. Cui, "A multilevel optimization framework for computation offloading in mobile edge computing," *Math. Problems Eng.*, vol. 2020, no. 1, 2020, Art. no. 412479.
- [23] Y.-T. Chuang et al., "A real-time and ACO-based offloading algorithm in edge computing," *J. Parallel Distrib. Comput.*, vol. 179, 2023, Art. no. 104703.
- [24] L. Geng, H. Zhao, J. Wang, A. Kaushik, S. Yuan, and W. Feng, "Deep reinforcement-learning-based distributed computation offloading in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12416–12433, Jul., 2023.
- [25] C. Li, K. Jiang, Y. Zhang, L. Jiang, and S. Wan, "Deep reinforcement learning-based mining task offloading scheme for intelligent connected vehicles in UAV-Aided MEC," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–29, 2024.
- [26] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.
- [27] P. Hou et al., "Distributed DRL-based intelligent over-the-air computation in unmanned aerial vehicle swarm-assisted intelligent transportation system," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34382–34397, Nov. 2024.
- [28] C. Li, K. Jiang, G. He, F. Bing, and Y. Luo, "A computation offloading method for multi-UAVs assisted MEC based on improved federated DDPG algorithm," *IEEE Trans. Ind. Inf.*, vol. 20, no. 12, pp. 14062–14071, Dec. 2024.
- [29] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent DRL for task offloading and resource allocation in multi-UAV enabled IoT edge network," *IEEE Trans. Netw. Serv. Manage.*, vol. 18, no. 4, pp. 4531–4547, Dec. 2021.
- [30] H. Hu, G. Huang, X. Li, and S. Song, "Meta-reinforcement learning with dynamic adaptiveness distillation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 3, pp. 1454–1464, Mar. 2023.
- [31] B. A. Adoum et al., "Federated meta-learning for task offloading and resource allocation in MEC-IoT," in *Proc. Int. Conf. Radar Antenna Microw. Electron. Telecommun.*, 2023, pp. 122–128.
- [32] H. Huang, A. V. Savkin, and C. Huang, "Decentralized autonomous navigation of a UAV network for road traffic monitoring," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 57, no. 4, pp. 2558–2564, Aug. 2021.
- [33] V. Teeda, S. Moro, D. Scazzoli, L. Reggiani, and M. Magarini, "Differentiation and localization of ground RF transmitters through RSSI measures from a UAV," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 1000–1009, Jan. 2025.
- [34] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018.
- [35] J. Zhang et al., "Computation-efficient offloading and trajectory scheduling for multi-UAV assisted mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2114–2125, Feb. 2020.
- [36] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop multi-task partial computation offloading in collaborative edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1133–1145, May 2021.
- [37] D. P. Kingma et al., "Auto-encoding variational bayes," in *Proc. Int. Conf. Learn. Representations*, 2014, pp. 1–14. [Online]. Available: <https://doi.org/10.48550/arXiv.1312.6114>
- [38] Y. Li, J. Li, Z. Lv, H. Li, Y. Wang, and Z. Xu, "GASTO: A fast adaptive graph learning framework for edge computing empowered task offloading," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 932–944, Jun. 2023.
- [39] F. Song et al., "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023.
- [40] 2014. [Online]. Available: <https://ieee-dataport.org/open-access/yawdd-yawning-detection-dataset>
- [41] 2016. [Online]. Available: <https://www.kaggle.com/datasets/rightway111/state-farm-distracted-driver-detection>
- [42] S. A. Osia et al., "A hybrid deep learning architecture for privacy-preserving mobile analytics," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4505–4518, May 2020.
- [43] J. Wicaksana et al., "FedMix: Mixed supervised federated learning for medical image segmentation," *IEEE Trans. Med. Imag.*, vol. 42, no. 7, pp. 1955–1968, Jul. 2023.
- [44] Z. Bing, L. Knak, L. Cheng, F. O. Morin, K. Huang, and A. Knoll, "Meta-reinforcement learning in nonstationary and nonparametric environments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13604–13618, Oct. 2024.

- [45] A. Gupta, A. Trivedi, and B. Prasad, "B-GWO based multi-UAV deployment and power allocation in NOMA assisted wireless networks," *Wireless Netw.*, vol. 28, no. 7, pp. 3199–3211, Jul. 2022.
- [46] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.



Yong Zhang received the MS degree from Tiangong University, in 2020. He is currently working toward the PhD degree with the School of Computer Science and Technology, Wuhan University of Technology. His research interests include cloud computing and artificial intelligence.



Chunlin Li received the BS and MSc degrees in computer science from the Wuhan University of Technology (WUT), China, in 1996 and 2000, respectively, and the PhD degree in computer software and theory from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2003. She is currently a professor and PhD tutor of computer science with WUT. Her research interests include edge computing.



Chaoyue Deng received the bachelor's degree from the Wuhan Institute of Technology, in 2022. He is currently working toward the MS degree with the School of Computer Science and Technology, Wuhan University of Technology. His research interests include cloud computing and artificial intelligence.



Shaohua Wan (Senior Member, IEEE) received the PhD degree from the School of Computer, Wuhan University, Wuhan, China, in 2010. He is currently a full professor with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen, China. From 2016 to 2017, he was a visiting professor with the Department of Electrical and Computer Engineering, Technical University of Munich, Munich, Germany. He is the author of more than 150 peer-reviewed research papers and books, including more than 50

IEEE/ACM Transactions papers, such as *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Wireless Communications*, *IEEE Transactions on Communications*, *IEEE Transactions on Intelligent Transportation Systems*, *ACM Transactions on Internet Technology*, *IEEE Transactions on Network Science and Engineering*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Multimedia*, *ACM Transactions on Embedded Computing Systems*, *ACM Transactions on Multimedia Computing, Communications, and Applications*, *IEEE Transactions on Automation Science and Engineering*, and many top conference papers in the fields of edge intelligence. His research interests mainly include deep learning for intelligent transportation systems.